Dynamic Statistical Optimization of Disk Access in a Virtualized Environment

Andrew Cassidy

University of Kent

ajc39@kent.ac.uk

Abstract

This report is the description and results of the proposed implementation of a transparent disk shuffling framework. This framework has been implemented and tested with a particularly simple Working Set style algorithm, mostly using technologies, such as the Logical Volume Manager (LVM) and the Block Layer IO Tracing system. In this report the framework produced is evaluated and, and future work based on this is discussed.

Introduction

Virtualization, a system used to run multiple operating systems concurrently on a single physical machine, is increasingly used to leverage available computing power, sharing the resources of a single physical system between multiple virtualized computing environments, primarily for reasons of cost. However, this increased sharing leads to increased contention. For the disk I/O and paging subsystems this can have a performance damaging impact, since the disk is typically orders of magnitude slower than memory. In a virtualized environment, this can result in a situation where many or all virtualized systems are stalled, waiting for requests on the underlying physical disk to complete. This is also compounded by the fact that the virtualized systems can only optimise the placement of data within their own environment, meaning that data cannot be placed as optimally on the shared physical disks as if they were all independent.

The primary aim of this project is to produce a framework to allow the rearrangement of the data on the disk, that will reduce block service times, whilst maintaining transparency to the upper levels of the device hierarchy. This should reduce seek time (the time taken for the heads to move to the location required), reducing the amount of time the system is waiting for disk IO operations to complete.

Section 2 provides some background on the various technologies involved in this framework, including LVM and Xen. The approach and implementation of the framework is described in section 3, followed by an initial evaluation of benchmark results in section 4. Section 5 presents some concluding remarks, followed by a discussion of related and future work in sections 6 and 7 respectively.

Background

Traditionally, the response to this problem has been to purchase more disks. This option is generally cheaper than researching into optimisation methods, as the price of modestly-sized disks has become less than two days wages for an average intern, and solves the problem by adding more available heads to access the disks concurrently. This approach is not always possible, as eventually the available IO connectivity, such as SATA ports, will eventually become exhausted.

Logical Volume Manager

Introduction

The Linux Logical Volume Manager (LVM) [1] is a block storage abstraction layer allowing multiple disks to be treated as one large device, that can be split into any number of smaller logical volumes. This allows for greater flexibility of the utilisation of available storage space, and is often used in conjunction with the Xen hypervisor which is described in section 2.3 of this report. This is because in older versions of Xen each virtual machine required its own block devices.

Architecture

LVM is built up of several layers:

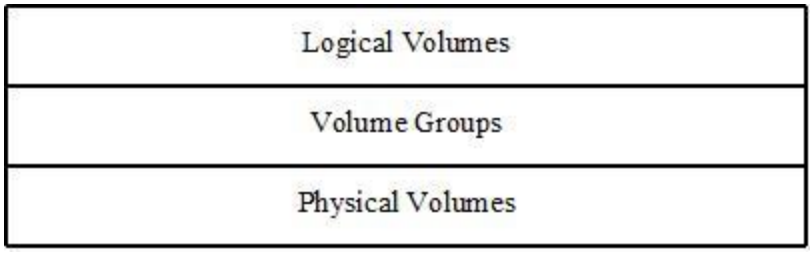


Fig 1. Basic LVM Layers

The physical and logical volumes are split up into "Extents", fixed size blocks of data, typically 4MB. Logical Extents can be mapped to one or more Physical Extents, allowing for Logical Extents to be mirrored or even striped across Physical Volumes. Which physical extent they are mapped to is not important, and these mappings can be changed. Physical Volumes can either be entire disks, or individual partitions, allowing for more flexible use. For instance, you could have a volume group that mirrors data across a number of disks, whilst another stripes or spans data across disks. Here is an example LVM configuration, allowing 3 logical volumes to be spread across 4 physical volumes transparently to the file system.

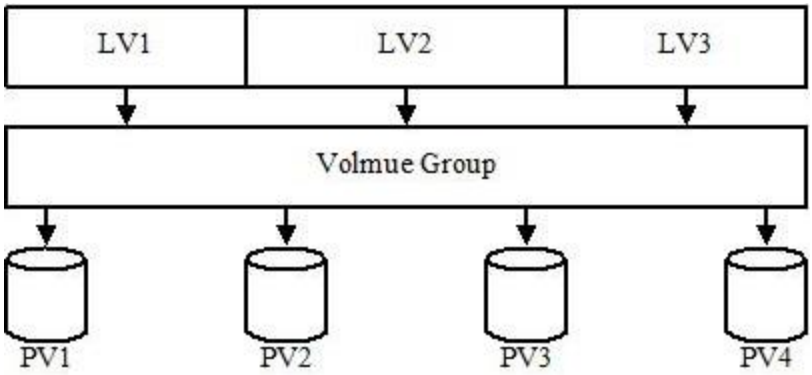


Fig 2. An example LVM configuration

Each physical volume consists of a header which is usually the first 192KB of the volume, and contains all the metadata for the entire Volume Group, and is replicated across all physical volumes for redundancy. The physical extents begin directly after this, and are sequentially numbered from 1. These numbered addresses are what is needed in order to move physical extents in the fashion mentioned below. Note that it is only the data within the physical extent that is moved, and the logical extent will be remapped to reflect the new address of the data.

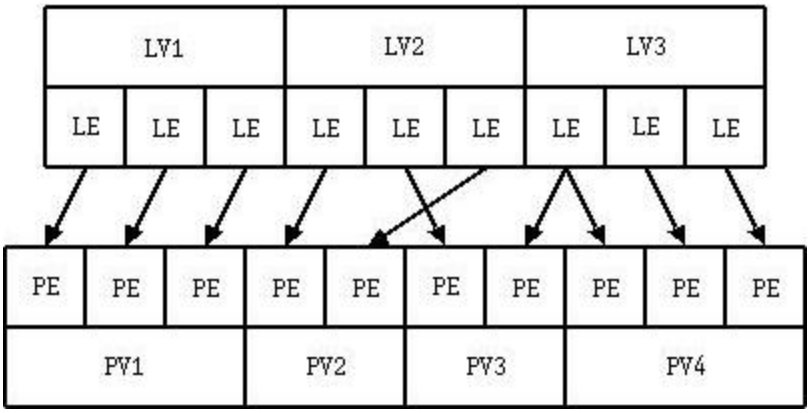


Fig 3. An example mapping of Logical Extents to Physical Extents

These ideas bring about 3 emergent properties:

- Extents can be mirrored across multiple physical volumes.
- Extents can be distributed across multiple disks, allowing for striping of volumes.
- Physical Extents can be moved whilst the volume is online, completely transparently to the logical volumes.

It is the latter of these that make rearranging data possible.

Moving Physical Extents

Moving physical extents requires the destination extent to be an unallocated Physical Extent in the same volume group. The process of moving physical extents is as follows:

1. The destination extent is allocated as a mirror of the source extent
2. The extent is copied
3. The Logical Extent mapping is updated
4. The mirror is destroyed
5. The source extent is deallocated

This is handled by the command line tool 'pvmove', which takes the source extent path and an optional destination extent path. An example pvmove invocation:

```
pvmove /dev/sda:4-10 /dev/sdb
```

Where /dev/sda and /dev/sdb are Physical Volumes, and extent numbers 4 though 10 are to be moved to /dev/sdb. Where destination extent numbers are not specified, the physical extents are allocated as per LVM's default allocation rules.

Blktrace

Block Layer IO Tracing was added into the Linux kernel in 2005 by Jens Axboe. Much unlike the iostat utility already available, it provides per IO operation, per device and per CPU tracing at every stage in the lifetime of the request. This includes merges, where requests for the same block or adjacent blocks are merged into once large request to reduce both rotational delay and seek times. This is implemented at the block layer, and hooks directly into the request queues [2].

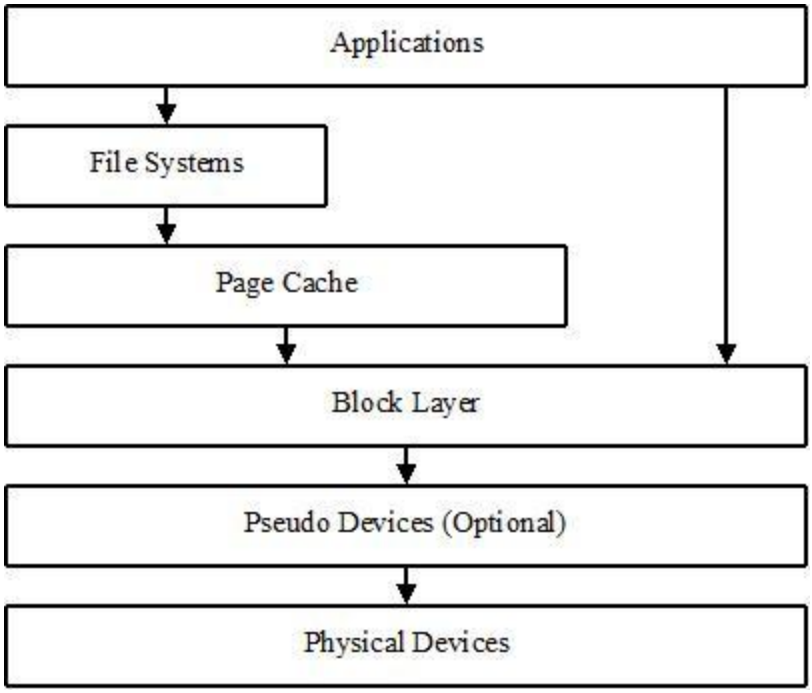


Fig 4. Simplified Block Device Layers

Xen Hypervisor

The Xen Hypervisor [3] is a so-called "bare-metal" visualization system. Running directly on the hardware, Xen is in effect the host operating system. Xen manages virtual machines as domains, which usually run a modified Linux kernel, but can run unmodified "guest" operating systems on systems with support for hardware virtualization, including most new processors such as the Intel Core2 range.

Xen cannot run by itself, and what would normally be known as the host OS is loaded on boot into the first domain, referred to as "dom0". Xen uses this domain to manage the available system resources and configure other domains, referred to a "domU". It also means that requests to hardware are redirected through this domain, allowing for resources to be centrally managed.

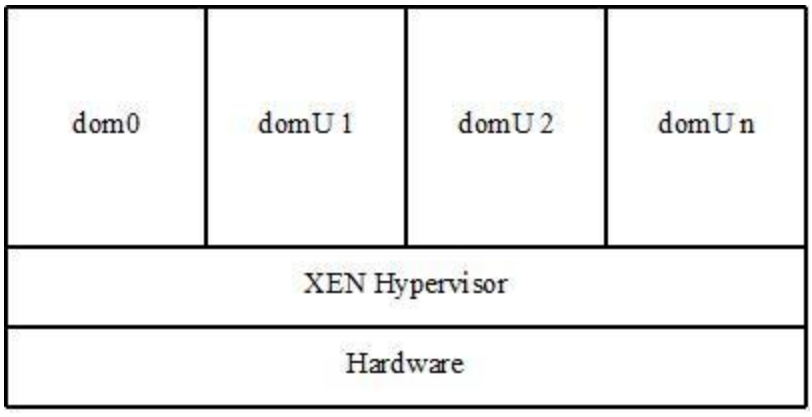


Fig 5. Basic XEN Structure

Xen manages conflicts for resources such as disk space and physical memory by not allowing these resources to be over-allocated. Each domain's configured disk space and memory is guaranteed and preallocated. However this does not cover disk I/O bandwidth, which this project seeks to address.

Although recent versions of Xen support for using files via the blktap driver, a pluggable block device driver allowing Xen guests to use a number of other devices as virtual disks, it is still commonplace to use LVM Logical

Volumes to use a the root block devices of Xen guests.

Approach

Making use of the technologies mentioned above, and the features they provide, it is possible to implement a transparent, continuous data reorganisation system. The related works not only demonstrate that this is possible, but that they can produce performance increases.

Using a simple time stamp file alongside the blktrace logging and the transparent data moving features of LVM, it should be possible to implement any number of data replacement strategies. With these ideas in hand, the system is intended to be extendable, allowing developers and system administrators to create their own data replacement strategies more suited to their needs.

Replacement Algorithm

The data replacement algorithm to be used for the demonstration of the framework will be a simple working set algorithm. While less intelligent than the "organ pipe" system presented in the related work [5]. The continuous reorganisation should show improvements with minimal overheads. Taking the most recently used data and placing it in the middle of the disk and swapping out old data, should reduce head movements overall. To ensure that this process does not thrash the disks, a minimum age for swapping out will be imposed.

Disk Layout

The disk will be be partitioned into three, the central working set, and the two outer sets. The central working set shall be 50% of the disk, and the outer sets 25% each. On the creation of the logical volume to represent the disk, at least one physical extent must be left unallocated in order to facilitate the movement of data. This design meets the requirements for needing one unallocated physical extent in order to facilitate the moving of data, and minimises head movements outside of the working set (should data outside of it be requested). Note however, that due to the lack of a reliable source of CHS information on modern drives, that these locations on the disk are guessed at based upon sector number.



Fig 6. Disk Layout

Implementation

The system is to be implemented in C, in three sections. Firstly, a framework library including a main executable that will handle the configuration and volume detection for the whole system and provide access to various configuration information. The second component is a library of helper functions and data structures available for use in the replacement algorithms. These include accessor functions for the configuration, actually performing the data move, and resolving the sector numbers to physical extent addresses. The final component is the algorithm itself. In this instance, it will be the working set implemented above, but can be easily extended as long as the three function pointers are provided in main.c.

The main loop simply launch blktrace and reads the output, which is then parsed and passed on to the running algorithm[1]. The main loop can be terminated by handling a SIGINT.

The working set algorithm itself has its own configuration file to specify which part of the disk is to be the working set, and to create the timestamp map file if required. Upon receipt of a record from blktrace, the sector is resolved to a Physical Extent, and then checked if it resides in the working set. If it does, the timestamp file is updated to reflect this access. Otherwise, a search is performed to find a Physical Extent within the working set that is older than the specified minimum age to be swapped. The two PE's are then swapped, by moving the old one out of the working

[1]Due to time constraints. It should be possible to integrate blktrace properly in order to reduce the overheads presented by launching an external process.

set, and then the new one in.

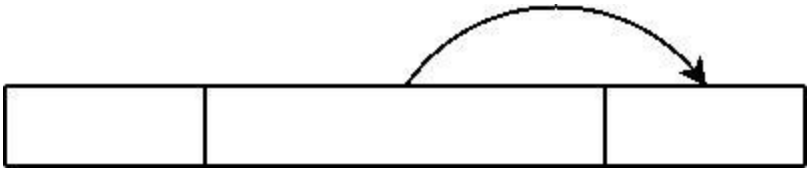


Fig 7. Moving old data out of the working set.

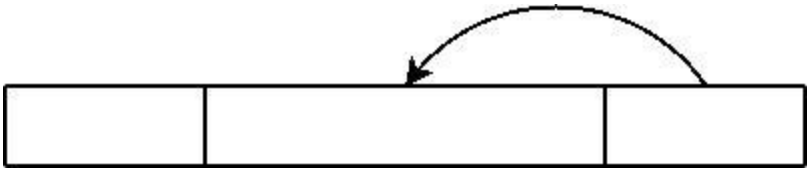


Fig 8. Moving recently accessed data in to the working set.

One of the major limitations caused by this shuffling is that recovering data from the shuffled disk may become difficult should the LVM metadata containing the LE to PE mappings become damaged, although this is unlikely as the metadata is mirrored across all physical volumes in the volume group. Also, where larger files are involved, it is likely that files will become fragmented, as the PE's they are present in could both be moved to opposite sides of the disk should they be swapped out of the working set, increasing the read time over the whole file.

Initial Testing

For the initial testing phase, the tool used was IOzone [http://www.iozone.org/], a file system benchmarking tool. The test machine was of the following specification:

- AMD X2 6000+ CPU
- 2GB DDR2 RAM
- 160GB Hitachi Travelstar Hard Disk
- Debian GNU/Linux 5 "Lenny" with LXDE

The following packages are required to build and run the system:

- build-essential
- gdb
- blktrace

The benchmark program, IOZone is available from the IOzone website [http://www.iozone.org/]. The version used was 3.2.1.

The tests, consisting of just a single run each due to time restrictions, will consist of IOzone run under the following conditions:

- Clean run
- Continuously reorganised using the program written for this project, using a minimum PE age of 10 minutes.
- Continuously reorganised using the program written for this project, using a minimum PE age of 30 minutes.

The timing for the tests could alos be done using the count of the IO processes, rather than wall clack time. This would mean that block are less likely to timeout before the next operation occurs over long periods of inactivity.

Whilst IOzone runs a large number of tests, the ones that will be most interesting are the re-read and re-write tests, which occur after the read and write tests respectively, as the data read or written will have been moved into the working set. Because of the amount of test data returned, the test results are aggregated to produce series suitable for graphing.

For all tests, the graph shows the average data throughput in kilobytes per second, plotted against the size of the file to be read or written, also in kilobytes.

Write Tests

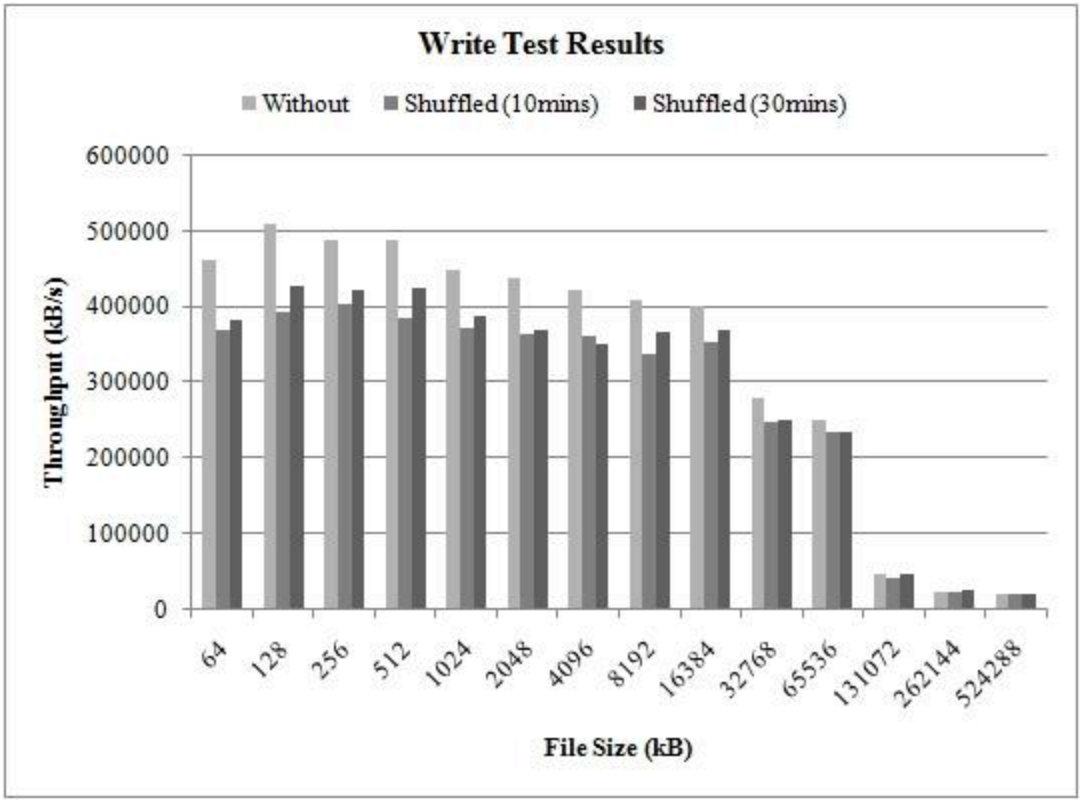


Fig 9. Graph of Write Test Results

Figure 9 shows that under continuous reorganisation the initial write of files are slower. This is as expected, as while the next test is being run, the data from the first test is being moved into the working set. This is depicted in Figure 10.

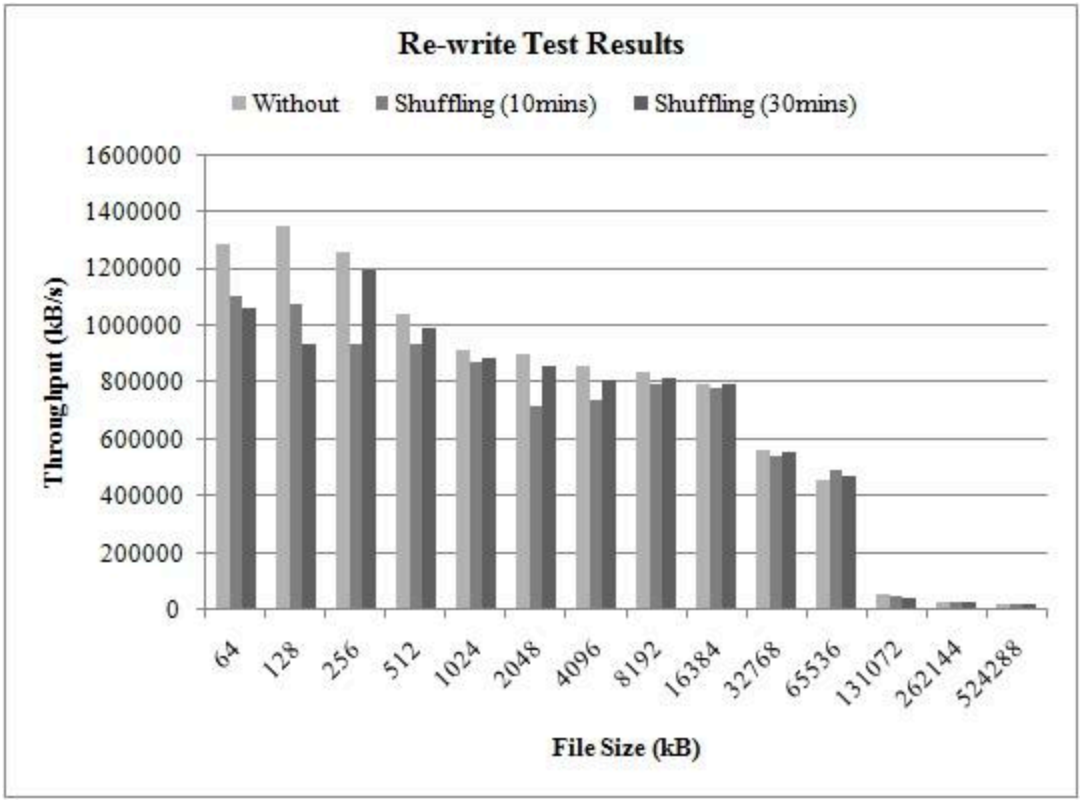


Fig 10. Graph of Re-write Test Results

However it seemed that this was not quite the case, and in most cases writes were instead still slower. This could be down to the data being written to disk while the data is still being moved.

Read Tests

Figure 11 shows the results of the read tests, for which there were the same expectations in terms of results.

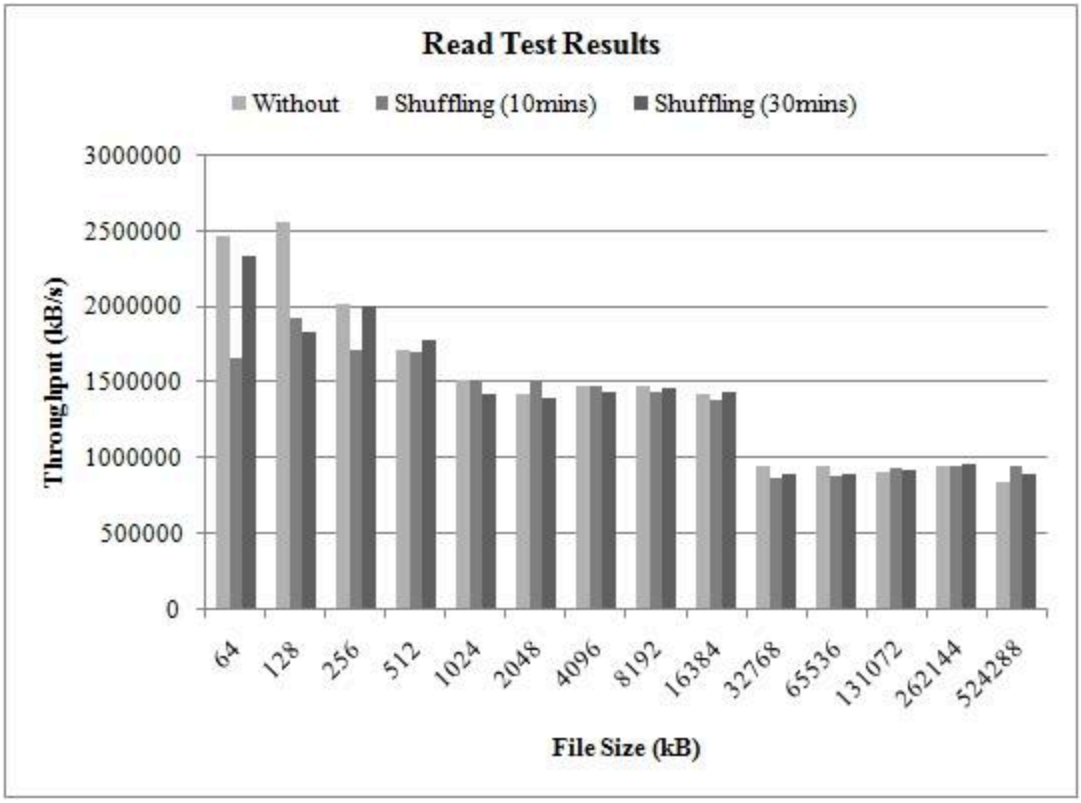


Fig 11. Graph of Read Test Results

Where in the write test performance was down across the board, in the read test, it appears that even whils't the disk is being shuffled, there is an increase in read speed for larger files. And the re-read produced similar results for the large files, but in more cases there were marginal performance increases.

Figure 12 shows improvements as soon as the 512k files, although this is not consistent across the entire range. Some of the larger files higher throughout could be because they have been cached in memory, and not being re-read from the disk.

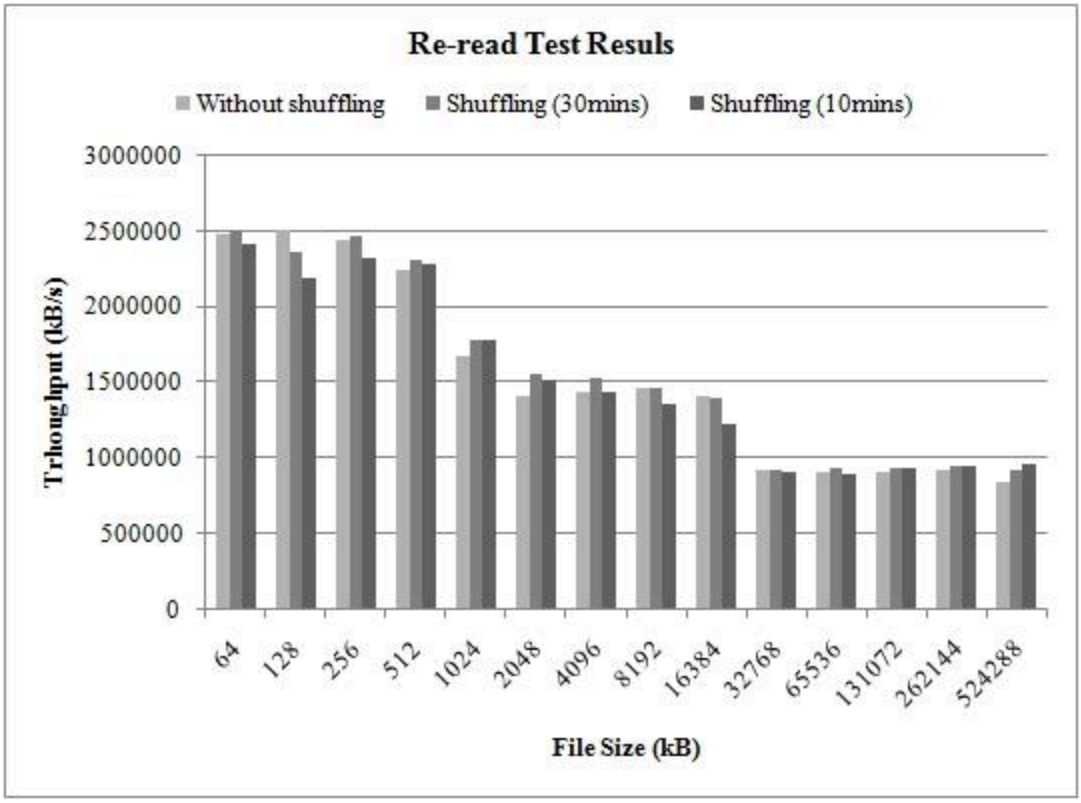


Fig 12. Graph of Re-read Test Results

Conclusions

Although the results are not spectacular, there are some signs that the general idea is working. In order to show more improvement, the tests would have to have been run on a live system for a lot longer, using real IO rather than generated. The longer the test is run, the more time is given for the reorganisation of the disk to settle out, and therefore the extra disk load produced from the reorganisation will gradually decrease.

There is something to be said about the current results, which strangely do show improvements on the re-read and re-write tests even though the disk is still under load from moving the data around. It could be said that there is a much more significant performance increase that has in these tests been negated by the continuous reorganisation, although these increases can possibly be attributed from reading from the in memory cache rather than the disk.

The 4MB extent size could also be another factor in some of the poor results. When the smallest file is moved, the whole 4MB extent is moved. This extent could potentially contain a parts of or even whole files. This could be fragmenting larger files, which increases the number of seeks to read the file. This extent size could also be leaving frequently used data out, as usable space in the working set could be being wasted by having a whole extent containing a single 4kB file.

Whilst the results shown are inconclusive for the specifics of the reorganisation, the overall project has been a success. It is evident that under the correct circumstances, it is indeed possible and feasible to optimise disk access in this manner, and this has been facilitated by the framework produced as part of this project.

Related Work

Recent work on optimising disk access and data placement has mostly been based on and targeted towards distributed storage systems, where the access patterns and requirements are significantly different to those of virtualized environments. Whilst these distributed storage systems are based around speed of access for a single logical volume across a number of disks or nodes, virtualization requirements are the complete opposite, providing multiple volumes on just a single physical disk.

There have been attempts at this, particularly HP. Ruemmler and Wilkes [5] attempted to move all data on a given disk to the middle in order to reduce head movements and seek times, based upon the 4.2BSD file system, in which the performance improvement was minimal, 2-15%.

This work, however is not transparent to the file system, and the approach is unsuitable for a virtualized environment, where the file systems in question are isolated. Although the performance improvements were minimal, this idea could still aid the performance of a saturated system.

Salmon, Thereska, Soules and Ganger [4] had a similar approach at the file system level, but splitting the reorganisation into another layer. Their approach also differed in the algorithm used in order to place the data, taking a more heuristical and constraint-based approach. Their results showed a significant performance increase based on continuous reorganisation of data over basic shuffling towards the beginning of the test, but they seemed to level out as time progressed. Overall both the shuffled approach and constant reorganisation produced better response times to the unmodified disk.

The approach taken by English and Stepanov [6] however, fits the virtualization problem better by doing the data organisation and mapping at a controller level, examining the track records of the previous file system and software approaches. The controller, codenamed "Loge", places data based primarily on the relationship between the current head position and the available space. The paper indicates that future work on the controller could include continuous reorganisation, noting that being a hardware solution, the ability to detect a system's idle state is somewhat easier than in a software solution. They also indicate that the enhancement to performance may be negated entirely by the overheads associated with moving the data.

Since the HP papers [5, 6] were written, however, there have been many advances in hard disk technologies, including Native Command Queuing and Zone Bit Recording. The former allows a hard disk to reorganise its own request queue to reduce rotational delay, and the latter allows more data to be stored on the less dense outer tracks of disk. ZBR has also invalidated the CHS disk structure, and the mapping between Logical Block Allocation numbers and their location on the disk is now mapped by the hard disk itself, and the CHS is provided only for compatibility with legacy operating systems. The other paper [4] was written around the same time as Native Command Queuing papers were being published.

Future Work

The work that has been undertaken so far has been pretty broad-minded, in order to allow the framework to be used in many other experiments that there was insufficient time to complete. For the most part, default parameter values were used, and a very simplistic model for the access of data was used. However, other algorithms can be tested, using some form of heuristics to determine where to place data based on the characteristics of data accessed before and after it. Also, the PE size of the LVM can be modified, maybe made smaller to take into the account that not many files are not anywhere near to the default 4MB size, although in general, the files on a Linux files ystem will generally be spread out across the whole disk. Furthermore, a more intelligent approach could be taken to choosing PE's to move in to the working set, based on the frequency of the access rather than just the last access time. This current approach moves data into the working set, even if it is just a one off read.

Another point worth noting is that the working set algorithm implemented merely searched for and swapped the first PE it found that was above the minimum age, meaning there was some bias to towards PE's at the beginning. This decision was made because an exhaustive search for the oldest PE was rather time consuming as it scanned the whole file, although it may also be feasible to load the entire map into memory, writing back at set intervals. Another option is to load the entire file in as some form or modifiable priority queue, giving a higher priority to swap out older PE's.

Another issue presented whilst running the tests it that currently all processing is done on a single thread, and the data move is processed synchronously. Future work could be done to process the move on another thread, allowing the raw data to continue to be processed whilst the data is being moved.

Acknowledgments

I'd like to thank the project supervisor, Fred Barnes for his support throughout the project, especially for locating good reading materials that helped the initial stages of the project, along with proof reading and general advice on

the production of this paper. Thank you to Tom Carlson, for help preparing this paper, and to W.F. Konynenberg, ContextShift Ltd, for proposing this project.

Bibliography

[1] Anatomy of LVM. The Linux Documentation Project. http://www.tldp.org/HOWTO/LVM-HOWTO/anatomy.html (Accessed Accessed 21st March 2009)

[2] A. D. Brunelle. Block I/O Layer Tracing: blktrace http://www.gelato.org/pdf/apr2006/gelato_ICE06apr_blktrace_brunelle_hp.pdf (Accessed 21st March 2009)

[3] Xen Architecture Overview http://wiki.xensource.com/xenwiki/XenArchitecture?action=AttachFile&do=get &target=Xen+Architecture_Q1+2008.pdf (Accessed 21st March 2009)

[4] B. Salmon, E. Thereska, C.A.N. Soules, G.R. Ganger. A Two-Tiered Software Architecture for Automated Tuning of Disk Layouts. *Published In The Proceedings of the Federated Computing Research Conference (FCRC). San Diego CA. June 11, 2003.*

[5] C. Ruemmler, J. Wilkes. Disk Shuffling. *http://www.hpl.hp.com/research/ssp/papers/HPL-91-156.pdf* October, 1991 (Accessed 21st March 2009)

[6] R.M. English, A.A. Stepanov. Loge: a self-organizing disk controller. *Published in Proceedings of USENIX Winter 1992 Technical Conference, January 20–24, 1992 (San Francisco,CA), pp 237–251.*